the Razor Page; for MVC controllers, the view must be located at runtime.

Some of this figure should be familiar; it's the bottom half of figure 19.3 (with a couple of additions). It shows that the MVC controller action method uses a `ViewResult` object to indicate that a Razor view should be rendered. This `ViewResult` contains the name of the Razor view template to render and a view model, an arbitrary plain old CLR object (POCO) class containing the data to render.

NOTE `ViewResult` is the MVC equivalent of a Razor Page's `PageResult`. The main difference is that a `ViewResult` includes a view name to render and a model to pass to the view template, while a `PageResult` always renders the Razor Page's associated view and always passes the `PageModel` to the view template.

After returning a `ViewResult` from an action method, the control flow passes back to the MVC framework, which uses a series of heuristics to locate the view, based on the template name provided.

Once it locates the Razor view template, the Razor engine passes the view model from the `ViewResult` to the view and executes the template to generate the final HTML. This final step, rendering the HTML, is essentially the same process as for Razor Pages.

You can add a new Razor view template to your application in Visual Studio by right-clicking the folder you wish to add the view to in Solution Explorer. Choose **Add** > **New Item** and then select **Razor View - Empty** from the dialog, as shown in figure 19.5. If you aren't using Visual Studio, create a blank new file in the Views folder with the file extension .cshtml.